オペランド抽象化と n-gram 特徴量に基づく難読化バイナリ関数の検知

奥野 竜太[1,a)]　青木 茂樹[1,b)]　宮本 貴朗[1]

概要：難読化バイナリの検知は，マルウェア解析における初期トリアージや解析手法選定の効率化において重要である．従来手法として制御フローグラフ（CFG）に基づく GNN 手法やオペコード統計手法が存在するが，前者は計算コストの高さ，後者はオペランド情報の欠落により詳細な挙動を把握できない点が課題であった．本稿では，これらの課題を解決するため，役割ベースのオペランド抽象化に基づく軽量な非 GNN 手法を提案する．提案手法では，アセンブリ命令のオペランドをその役割に応じて抽象化してオペコードと結合し，n-gram および TF-IDF を用いて局所パターンと出現統計を特徴量化する．分類器には XGBoost を採用する．評価実験では，難読化ツールのバージョン（Tigress 3 および 4）やコンパイラ（GCC および Clang）が異なる Out-of-Distribution（OOD）環境下で検証を行った．損失関数によるクラス不均衡補正を適用した評価の結果，提案手法は多くの環境で既存手法を上回り，環境変化が大きい条件においても同等以上の高い汎化性能と堅牢性を示すことを確認した．

キーワード：難読化検知，オペランド抽象化，n-gram，バイナリ解析

Detection of Obfuscated Binary Functions Using Operand Abstraction and n-gram-Based Features

Ryuta Okuno[1,a)]　Shigeki Aoki[1,b)]　Takao Miyamoto[1]

Abstract: Detecting obfuscated binary functions is vital for streamlining malware triage and selecting analysis techniques. Existing CFG-based GNN and opcode-statistic methods suffer from high computational costs or fail to capture fine-grained behaviors due to missing operand information. To address these limitations, we propose a lightweight non-GNN method leveraging role-based operand abstraction. Our approach abstracts operands by role, combines them with opcodes, and extracts local patterns and occurrence statistics using n-grams and TF-IDF for XGBoost classification. We evaluate the method under Out-of-Distribution (OOD) settings across different obfuscation tool versions (Tigress 3/4) and compilers (GCC/Clang). Class-imbalance-weighted evaluations demonstrate that our approach outperforms existing methods in most environments and maintains comparable or superior robustness and generalization even under severe environmental changes.

Keywords: Obfuscation Detection, Operand Abstraction, n-gram, Binary Analysis

1. はじめに

難読化とは，プログラムの機能を変えずにコードを複雑化させる技術であり，制御フローの複雑化や圧縮・暗号化など多様な手法が存在する．難読化は，商用ソフトウェアにおいて重要なアルゴリズムや認証情報をリバースエンジニアリングから保護する目的で利用される一方 [1]，マルウェアがセキュリティ製品の検知を回避し，解析を妨害する手段としても悪用される [2]．コードの難読化に伴う制御構造やデータ操作の変形は，静的解析の精度低下や解析

[1] 大阪公立大学大学院情報学研究科
Graduate School of Informatics, Osaka Metropolitan University
[a)] sj22603x@st.omu.ac.jp
[b)] aoki@omu.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

時間の増大を招き，対策策定や解析結果の信頼性に悪影響を及ぼす．したがって，事前に難読化の有無を検知することは，適切な解析手法の選定や優先解析関数の特定といった初期トリアージにおいて有用である．

文献 [3] では，バイナリプログラムの制御フローグラフ（CFG）に基づく GNN（Graph Neural Network）を用いた関数単位の難読化検知手法が提案されている．同文献では，ベースラインである Gradient Boosting とオペコード特徴量に基づく手法が，一部の GNN 手法と同等の精度を示すことが報告されている．この結果は，CFG の基礎となるアセンブリ命令列に難読化検知のための有用な情報が含まれていることを示唆している．また先行研究 [4] においても，`jmp` などの制御フロー命令や `cmp` などの論理演算命令といった特定役割のアセンブリ命令群が検知の手がかりとなることが示されている．しかし，従来のオペコード特徴量は命令の種類のみを扱うため，オペランドやデータ操作といった難読化に伴う多様な変化を，CFG の構造的特徴ほど十分には反映できない．特に，`call` や `mov` などの汎用的なオペコードで実装された難読化の挙動は，オペコード単体では捕捉が困難である．

本稿では，バイナリプログラム解析に基づき，難読化を識別する軽量な非 GNN 手法を提案する．提案手法では，抽出したアセンブリ命令列のオペランドを役割ベースで抽象化してオペコードと結合し，n-gram と TF-IDF を用いて局所パターンを特徴ベクトル化する．分類器には XGBoost[5] を採用する．一般に難読化は実行速度の低下を伴うため，実用上は隠蔽対象の関数のみに適用されることが多い．実際に，Tigress[6] などの一般的な難読化ツールは関数指定での難読化に対応している．そのため，本研究では検知を関数単位で行う．評価実験では，ツールのバージョン変更（Tigress 3 から Tigress 4）やコンパイラ変更（GCC から Clang）を含む Out-of-Distribution（OOD）環境下におけるロバスト性を検証する．

以下，第 2 節で関連研究を紹介し，第 3 節で提案手法を述べる．第 4 節では実験結果および考察を示し，第 5 節でまとめを述べる．

2. 関連研究

本研究に関連する従来研究として，制御フローグラフ（CFG）に基づく GNN 手法 [3]，およびアセンブリ命令列を Word2Vec によりベクトル化して深層学習モデルで解析する手法 [7] について述べる．

文献 [3] は，CFG を用いてプログラムをグラフ表現し，GNN により難読化の検知・分類を行う手法を提案している．CFG はプログラムの制御遷移を表す有向グラフであり，各ノードは分岐を含まない連続命令列である基本ブロック（BB: Basic Block），エッジは条件分岐やジャンプなどの遷移を表す．同文献では，ニーモニックに基づく 128 次元の TF-IDF 特徴量を用いたベースライン手法が，一部の GNN 手法と同等の精度を示すことが報告されている．同文献では，「粗いアセンブリ表現に基づくグラフレベルの特徴量よりも，アセンブリ命令の分布を細粒度に表現するほうが，難読化により生じる異常性を的確に捉えられる」と考察されており，詳細な命令情報を扱うことの有効性が示されている．

文献 [7]（OBFEYE）は，Word2Vec の Skip-gram モデルを用いてアセンブリ命令列を命令同士の類似性を反映した数値ベクトル（分散表現）へ変換し，CNN による BB 内の局所特徴抽出と LSTM による関数全体の長距離依存関係の学習を行う手法である．この手法は，`ControlFlowFlattening` のように制御フローが構造的に変化する難読化に対して高い精度を示す．一方で，文脈情報を主に扱う設計であるため，`EncodeArithmetic` のように計算式の等価変換や即値書き換えを伴うデータフロー中心の難読化に対しては十分に対応できない課題が存在する．

以上の知見から，難読化検知には制御フロー構造のみならず，命令列レベルの統計的特徴や局所パターンの捕捉が不可欠であると考えられる．本研究では，文献 [3] が示した細粒度な命令情報の有用性に基づき，オペコードに役割抽象化したオペランドを結合した命令表現を採用する．さらに，文献 [7] とは異なり TF-IDF による統計的特徴量を用いることで，制御フローに依存しないデータフロー型の難読化挙動も効果的に捉える．これらの特徴量設計により，制御フロー型およびデータフロー型の双方に対して頑健な検知手法を構築する．

さらに，既存研究の多くは同一の難読化ツールバージョンおよび同一コンパイラ環境下で評価を行っており，特定のツールやコンパイラ固有のパターンへ過剰適合している懸念がある．この問題を検証するため，本稿ではツールバージョンやコンパイラが異なる Out-of-Distribution（OOD）環境下における評価を行う．

3. 提案手法

本手法では，バイナリプログラムからアセンブリ命令列を抽出し，オペランドを役割ベースで抽象化した命令列に対して n-gram を適用する．そして，得られた局所パターンを TF-IDF により特徴量へ変換して解析を行う．**図 1** に提案手法の概要を示す．

3.1 特徴量抽出・コーパス作成

本研究では，Ghidra[8] などの逆アセンブラにより得られた解析結果から各関数のアセンブリ命令列を抽出する．既存手法で一般的なオペコードのみの特徴量抽出では，データ操作や制御フローの細かな変化を捉えきれない課題がある．そこで本手法では，各命令のオペランドをその「機能的役割」に基づいて抽象化し，オペコードと結合して単

© 2026 Information Processing Society of Japan

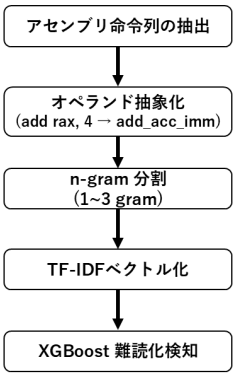


図 1 提案手法の全体概要
Fig. 1 Overview of the proposed method

表 1 命令オペランドの抽象化ルール
Table 1 Operand Abstraction Rules

記号	カテゴリ	該当例・判定条件
mem	メモリ	[ ] や ptr を含む参照（例: [rbp-0x8]）
zero	ゼロ値	0, 0x0, 0.0
acc	アキュムレータ	rax, eax, ax, al, ah
arg	引数レジスタ	rdi, rsi, rdx, rcx, r8, r9 系統
reg	その他汎用	rbx, rbp, rsp, r10〜r15 系統
vreg	ベクトル	xmm0〜31, ymm0〜31, zmm0〜31, mm0〜7
imm	即値	16 進数・数値（例: 0x10, 42）
oth	その他	上記以外のオペランド

一のトークンとして表現する「オペランド抽象化」を導入する．

アセンブリ命令中のオペランドは**表 1** に示す定義に従い，以下のような役割カテゴリへ変換される．

- **レジスタの役割別抽象化:** 汎用レジスタ（reg），演算結果や返り値に使われる累積器（acc），関数引数受渡用レジスタ（arg）など，用途に応じて細分化して抽象化する．
- **メモリおよび即値の抽象化:** メモリアドレス参照（mem）や，定数・フラグ等の即値（imm）へ集約する．

例えば，add rax, 4 は add_acc_imm へ，mov [rbp-8], rdi は mov_mem_arg へ変換される．このオペランド抽象化により，コンパイラの違いやレジスタ割り当てのランダム性，即値の具体値といった表層的な命令表現の差分を吸収し，難読化の本質的な命令パターンを保持しつつ語彙数の爆発を抑制できる．

次に，この抽象化された命令（例：add_reg_imm, xor_acc_acc）の系列を入力とし，コーパスを構築する．**図 2** に示すように，命令列を 1-gram，2-gram，および 3-gram に分割して語彙化する．難読化処理では分岐条件の挿入や等価変換によって特徴的な命令パターンが連続して挿入されるため，n-gram を適用して局所的な命令列構造を捕捉する．なお，計算量を抑えつつ識別力の高い頻度情報を保持するため，出現頻度上位 5,000 語を最終的な語彙として採用する．

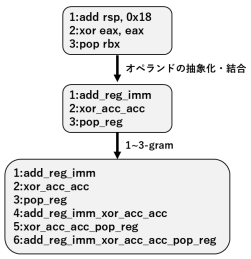


図 2 命令列の前処理
Fig. 2 Preprocessing of Instruction Sequences

3.2 特徴ベクトル生成

第 3.1 節で構築したコーパスに基づく n-gram 語彙を用いて，各関数の命令列を TF-IDF[9] による重み付き特徴ベクトルへ変換する．TF-IDF は，全関数で頻出する一般的な命令パターンの重みを抑え，特定の関数に特有のパターンを強調できるため，難読化によって局所的に挿入された命令群を捉えるのに適している．また，文脈構造に過剰適合しない統計的特徴量であるため，命令挿入や等価変換といったコード変換に対しても頑健に動作する．TF-IDF は次式で定義される．

$$\mathrm{TF}(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \tag{1}$$

$$\mathrm{IDF}(t) = \log \frac{N}{n_t} \tag{2}$$

$$\text{TF-IDF}(t,d) = \mathrm{TF}(t,d) \cdot \mathrm{IDF}(t) \tag{3}$$

ここで，$f_{t,d}$ は関数 d における語 t の出現回数，N は全関数数，n_t は語 t を含む関数数である．

TF-IDF の計算により，各関数は抽象化 1〜3-gram に基づく約 5,000 次元の数値ベクトルとして表現される．この「関数を文書，n-gram を単語」とみなす Bag-of-Words モデルにより，命令列中の局所的パターンを統計的に特徴づけた表現を抽出する．

3.3 難読化検知

本稿では，検知モデルとして XGBoost（eXtreme Gradient Boosting）[5] を採用する．XGBoost は，高次元かつ疎な TF-IDF 特徴量に対して頑健に動作し，非線形な特徴量の組み合わせを高精度に扱える利点がある．これにより，多様な難読化手法や最適化レベルが混在する場合でも，空間分割と正則化によって高精度に分類可能である．

また，データセットにおける難読化関数と非難読化関数の極端なクラス不均衡に対処するため，XGBoost のハイパーパラメータである scale_pos_weight にクラス比率に応じた重みを設定し，少数クラス（非難読化関数）の分類誤差に対するペナルティを強化した損失関数によりモデルの学習を行った．

© 2026 Information Processing Society of Japan

4. 評価実験と考察

4.1 データセット・評価指標

本手法の有効性を検証するため，文献 [3] で公開されている Diffing Obfuscation Dataset[10] を基にデータセットを作成した．本データセットは `zlib`，`lz4`，`minilua`，`freetype`，`sqlite` の難読化済みバイナリファイルおよびその解析結果で構成される．難読化には Tigress[6] を使用しており，文献 [3] に基づく難読化手法の分類は以下の通りである．

- **データ難読化:** `EncodeArithmetic`, `EncodeLiterals`
- **フロー難読化:** `OpaquePredicates`, `Flatten`, `Merge`, `Copy`
- **複合難読化:** `Flatten`，`EncodeArithmetic`，`OpaquePredicates` を組み合わせた `Mix1`，およびそれに `Split` を追加した `Mix2`

難読化ツールのバージョン（Tigress 3/4）やコンパイラ（GCC/Clang）の違いによる分布変化（Out-of-Distribution: OOD）に対する検知性能を評価するため，以下の実験設定を構築した．

- **学習データ（共通）:** Tigress 3 で難読化された C コードを GCC でコンパイルした関数（以下，`g3`）
- **テストデータ 1:** `g3`（In-Distribution 環境）
- **テストデータ 2:** Tigress 4 で難読化された C コードを GCC でコンパイルした関数（以下，`g4`）
- **テストデータ 3:** Tigress 4 で難読化された C コードを Clang でコンパイルした関数（以下，`c4`）

なお，学習環境とテスト環境の組み合わせは，学習環境とテスト環境の記号を並べて表記する（例：`g3` で学習し `c4` でテストする場合は `g3c4`）．

同一プロジェクトのデータが学習とテストに混在すると，プロジェクト固有のコーディングスタイルや命令分布へ過剰適合し，汎化性能を過大評価するリスクがあるため，データ分割においては `minilua` と `lz4` を学習データ，`zlib` を検証データ，`freetype` と `sqlite` をテストデータとして割り当てる構成とした．また，命令数が極めて少ない単純な関数によるノイズを抑えるため，ニーモニック数が 10 以下の関数はデータセットから除外した．

表 2 に示すように，本データセットは難読化関数が大半を占める不均衡データである．そのため，評価指標には多数派クラスの比率に左右されにくい Macro F1 スコアを採用し，各クラスの分類性能を適切に評価する．

4.2 比較手法

提案手法の優位性を検証するため，文献 [3] および文献 [11] の主要モデルを再現した 2 つの比較手法を設定した．

- **比較手法 1（手法 1）:** 文献 [3] に基づき，CFG の各基本ブロックをノードとし，ニーモニック出現頻度特徴量を割り当てて GCN（Graph Convolutional Network）により分類する手法である．同文献の記述に従い，Optuna を用いて隠れ層次元および層数のハイパーパラメータ探索を行った．
- **比較手法 2（手法 2）:** 文献 [11] に基づき，基本ブロックの統計的特徴量を GCN で集約した後，LSTM を通して時系列関係を学習する手法である．データセットの制約上，関数単位の文字列参照数を除外した 26 次元の特徴量を採用し，二値分類タスクへ拡張した．ハイパーパラメータは隠れ層次元 64 に固定し，学習時のクラス重み付き損失関数と検証データ上での Macro F1 最大化による分類閾値探索を独自に導入した．

予備実験において，有向グラフよりも無向グラフとして扱った方が安定して高い精度が得られたため，両比較手法の再現においては CFG のエッジを無向グラフ化して GCN へ入力する設計とした．この精度向上の要因としては，`ControlFlowFlattening` などの難読化によって形成される高次数のディスパッチャノードに起因する情報伝達の偏りを，無向化により軽減できたためと考えられる．

さらに，上記 2 手法に加え，最適化レベル（`O0`–`O3`）ごとに学習・検証・テストデータを分離してモデルを個別に構築する「提案手法（最適化レベル別）」も検証対象とする．これは最適化によるコード変化と難読化によるコード変化を明確に分離し，難読化特有のパターンを捉えやすくすることで精度向上を図るアプローチである．

表 2 各実験設定におけるデータセットの関数内訳
Table 2 Number of Plain and Obfuscated Functions in Each Experimental Setting

設定	データ	非難読化	難読化
g3g3	学習	2,345	33,978
	検証	270	4,622
	テスト	7,389	87,322
g3g4	検証	220	2,125
	テスト	4,895	44,508
g3c4	検証	211	1,780
	テスト	7,290	24,313

※ 学習データの対象ソースコードは共通であるが，最適化による関数の統合・削減や難読化の成否などにより，実際の学習データ件数は表中の数値（`g3g3`）から実験ごとにわずかに変動する．

4.3 実験結果

難読化検知の評価結果を**表 3** に示す．なお，表中のスコアはすべて 3 回の試行における平均値である．特に比較手法 1 の `g3c4` 環境や比較手法 2 の `g3g3` 環境では試行ごとの変動が大きく，いずれも最大で Macro F1 が 0.640 に達する試行が見られた．この不安定な挙動は，少数の検証データに過度に依存した分類閾値の探索や，ハイパーパラメータの選択が汎化性能に悪影響を及ぼした可能性が考え

© 2026 Information Processing Society of Japan

表 3 各実験設定における難読化検知性能の比較
Table 3 Performance Comparison of Obfuscation Detection Across Experimental Settings

設定	モデル	Macro F1
g3g3	提案手法	**0.726**
	提案手法（最適化レベル別）	0.716
	比較手法 1	0.690
	比較手法 2	0.602
g3g4	提案手法	**0.632**
	提案手法（最適化レベル別）	**0.632**
	比較手法 1	0.569
	比較手法 2	0.570
g3c4	提案手法	0.639
	提案手法（最適化レベル別）	0.624
	比較手法 1	0.629
	比較手法 2	**0.646**

表 4 最適化レベルごとの難読化検知結果
Table 4 Obfuscation Detection Results for Each Optimization Level

設定	最適化レベル	Macro F1
g3g3	O0	0.781
	O2	0.657
	O3	0.653
	Os	0.710
g3g4	O0	0.744
	O2	0.566
	O3	0.554
	Os	0.550
g3c4	O0	0.628
	O2	0.612
	O3	0.605
	Os	0.636

られる．

結果として，同一環境（g3g3）および難読化ツールのバージョンのみが異なる OOD 環境（g3g4）において，提案手法は両比較手法を明確に上回り，最も高い検知精度（Macro F1）を達成した．特に g3g3 において高い基礎性能を示しているだけでなく，g3g4 への環境移行に伴うスコアの低下幅が比較手法に比べて小さく抑えられており，バージョン変化に対する高い頑健性が確認された．

一方で，ツールバージョンとコンパイラの両方が異なる最も過酷な OOD 環境（g3c4）においては，比較手法 2（0.646）が僅差で最高値を記録したが，提案手法（0.639）もそれと同等の水準を維持しており，深刻な環境変化に対しても十分な汎化性能を有することが示された．

また，全体的な傾向として，ツールバージョンのみが変更された g3g4 よりも，バージョンとコンパイラの両方が変更された g3c4 においてスコアの低下幅が小さくなる現象が確認された．この要因としては，Clang のコンパイル規約や最適化に伴うコード生成の効率化・平滑化によって，非難読化（Plain）クラスにおけるコンパイラ固有の命令特徴が平準化され，誤検知（偽陽性）が大幅に減少したことが挙げられる．これにより非難読化クラスの F1 スコアが上昇し，結果として Macro F1 全体の低下が小さく抑えられたと考えられる．

4.4 考察

実験結果に基づく提案手法の有効性や，環境変化による影響，検知モデルの判断根拠（特徴量の重要度）について考察する．

4.4.1 提案手法の有効性と環境変化への頑健性

提案手法が g3g3 および g3g4 環境において安定して高い精度を記録した主な理由は，オペランドの役割抽象化とオペコードを結合したことで，オペコード単体では捉えきれなかった細粒度な命令表現を獲得できたためと考えられる．これにより，難読化特有の局所的な操作パターンを的確に捕捉可能となった．

一方，コンパイラとバージョンを同時に変更した g3c4 環境では，比較手法の精度が大きく上昇した．これは，Clang 特有のコード生成や最適化規約によって CFG のノード接続構造が標準的なパターンへ平滑化され，GCN のグラフ畳み込み処理に好影響を与えた可能性が考えられる．これに対し，提案手法が g3g4 から g3c4 への過酷な環境移行に対しても精度を損なうことなく（0.632 → 0.639），コンパイラの差異に依存しない安定した検知性能を維持できることが実証された．さらに，アセンブリ命令は細粒度で扱うべきであるとする先行研究 [3] の知見を裏付けるように，オペコード自体を抽象化せずにオペランドの役割のみを結合する本アプローチが妥当であることが確認された．

表 4 は，全最適化レベルのデータを一括で学習した提案手法（統合モデル）を用いて，テストデータを最適化レベル（O0, O2, O3, Os）ごとに分類して評価した結果を示している．分析の結果，未最適化（O0）の状態で最も高い検知精度が得られ，最適化レベルが高くなる（O2, O3）につれて精度が低下する傾向が見られた（ただし，g3c4 環境では Os が O0 をわずかに上回る）．これは，難読化によって意図的に挿入された冗長な処理や複雑な制御フローが，コンパイラの最適化過程で削減・単純化され，難読化特有のパターンが弱まるためであると考えられる．

また，最適化レベルごとに分離して学習させた「提案手法（最適化レベル別）」は，統合モデルを上回る精度の向上には至らなかった．この理由としては，XGBoost が最適化レベルの違いによる影響よりも，難読化によって挿入された特有の局所命令パターンを優先的に特徴量として捉

© 2026 Information Processing Society of Japan

表 5　各設定における上位 5 つの重要語彙

Table 5　Top 5 Important Features and Their Bias for Each Setting

設定	特徴量	バイアス
g3g3	lea_arg_mem_mov_arg_imm_call_imm	難読化
	ja_imm_movsxd_arg_mem	難読化
	mov_reg_arg_mov_arg_arg	非難読化
	mov_arg_mem_mov_acc_mem_cmp_arg_acc	難読化
	call_imm_mov_arg_reg_call_imm	難読化
g3g4	lea_arg_mem_mov_arg_imm_call_imm	難読化
	mov_reg_arg_mov_arg_arg	非難読化
	jmp_imm_leave_none_ret_none	難読化
	call_imm_mov_arg_reg_call_imm	難読化
	xor_acc_imm	難読化
g3c4	ja_imm_movsxd_arg_mem	難読化
	lea_arg_mem_mov_arg_imm_call_imm	難読化
	mov_reg_arg_mov_arg_arg	非難読化
	jmp_imm_leave_none_ret_none	難読化
	call_imm_mov_arg_reg_call_imm	難読化

えて判別を行ったためと考えられる．さらに，最適化レベルごとにデータを分割したことで各モデルが利用できる学習・検証データ件数が大幅に減少し，モデルの汎化性能やポテンシャルを十分に発揮できなかったことも精度向上の妨げとなったと推測される．

4.4.2　誤検知および難読化手法ごとの検知傾向

非難読化および難読化の誤検知・検知漏れについて分析を行った．非難読化関数の誤検知率については，g3g3 や g3g4 では最適化によって誤検知が増加する傾向にあったが，g3c4 では逆に最適化レベル O0 での誤検知が最も大きく，最適化後のほうが誤検知率が低下していた．これは GCC と Clang のコンパイラ特性やコード生成規約の違いに起因するものと考えられ，Clang における最適化処理によってコンパイラ間の出力命令の差異が平準化（効率化）されたことで非難読化関数の誤検知が抑制され，F1 スコアの向上に寄与したと推測される．

難読化手法ごとの検知率を分析すると，Flatten や EncodeArithmetic は全環境を通じて概ね高い精度で検知できた．これらの手法は制御構造を大きく改変したり，特徴的な計算ロジックを挿入したりするため，n-gram 特徴量での捕捉が容易であったと考えられる（ただし，Flatten 等も最適化による精度の低下は一部で確認された）．一方で，Copy や EncodeLiterals は相対的に低い検知率に留まった．特に Copy は，関数構造や命令列を改変せずにダミーの複製を作成する手法であるため，アセンブリ命令列の統計量や局所パターンにのみ依存する静的解析アプローチでは，本質的に非難読化コードと識別することが困難であるという限界が示された．

4.4.3　XAI に基づく特徴量の解釈と重要語彙の分析

モデルが判別に利用した重要な語彙（局所パターン）を分析するため，TreeSHAP[12] による特徴量重要度の上位 5 語を**表 5** に示す．SHAP（SHapley Additive exPlanations）は，機械学習モデルの予測に対する各特徴量の貢献度を割り当てる手法であり，モデルの判断根拠を説明可能にする XAI（Explainable AI）として広く用いられている [13]．表中の「バイアス」列は，その語彙が難読化/非難読化のどちらの判定に強く寄与したかを示している．なお，分析ノイズを排除するため，テストデータ全体の出現頻度が 1%以下の極端に低い語彙は除外した．

表 5 より，上位に 2-gram や 3-gram の語彙が多数ランクインしており，n-gram による特徴抽出が難読化特有の局所的な操作系列を的確に捉えられていることがわかる．また，実験環境（g3, g4, c4）の差異にかかわらず，call やジャンプ系の命令（jmp, ja），および演算命令（xor など）の組み合わせが難読化コードの強い判断材料として機能している．call や jmp を連続して含む語彙は Flatten（フロー平坦化）のディスパッチャ構造を，xor 等を用いた難解な計算の連続は EncodeArithmetic の特徴をよく捉えていると考えられる．

4.4.4　クラス不均衡に関する課題

本実験において，非難読化クラス単体の F1 スコアが相対的に低い値にとどまる傾向が見られた．この背景には，暗号処理などのように複雑なアルゴリズムを持つ非難読化関数がノイズとして存在することに加え，本データセットが「非難読化：難読化 $\approx 1:13$」という極端な不均衡データ（難読化データが 9 割以上）であるという構造的要因が挙げられる．少数クラスである非難読化クラスに対するわずかな偽陽性（False Positive）が，適合率（Precision）を相対的に大きく低下させるためである．マルウェア解析の初期トリアージという実用上の観点において，正常なプログラムを難読化と誤認するコストをいかに低減させるかは，今後の重要な課題である．

5.　まとめ

本稿では，バイナリプログラムから抽出したアセンブリ命令列に対し，役割ベースのオペランド抽象化とオペコードの結合を行い，n-gram および TF-IDF を用いて局所パターンを特徴量化する軽量な非 GNN 型の難読化検知手法を提案した．難読化ツールのバージョン差異やコンパイラ変更を含む 3 種類の評価環境下で検証を行った結果，提案手法は同一環境における高い検知精度を達成しただけでなく，Out-of-Distribution（OOD）環境下においても比較手法を上回る，あるいは同等の優れた汎化性能と堅牢性を示した．

一方で，最適化レベルごとにモデルを独立構築する「最適化レベル別モデル」による精度向上効果は限定的であっ

© 2026 Information Processing Society of Japan

た．これは，XGBoost が難読化特有の局所パターンを優先的に学習することに加え，データ分割に伴う学習サンプル数の減少が影響したためと考えられる．この結果は，事前処理による最適化レベルの同定やモデルの個別構築を行う必要がなく，全最適化レベルを一括学習した単一の統合モデルで十分な実用性を確保できることを示唆している．

今後の課題としては，パッキングや暗号化を含む未知の難読化手法に対する検知性能の向上，オペランド抽象化規則の高度化，および MSVC 等の多様なコンパイラ環境におけるロバスト性検証が挙げられる．特に，本研究での評価が Tigress に限定されている点を鑑み，実環境に存在する多様な難読化手法 [14] に対応するため，単なる命令列の出現統計にとどまらず，制御フローの構造変化やデータフローの変換を直接捉える特徴量の導入が不可欠である．さらに，マルウェア解析の初期トリアージにおける実用性を高めるため，プログラム本来の複雑な処理と難読化による意図的な処理を意味的（セマンティック）に識別し，誤検知率（偽陽性率）を抑圧する手法の構築を目指す．

参考文献

[1] Schrittwieser, S., Katzenbeisser, S., Kinder, J., Merzdovnik, G. and Weippl, E.: Protecting Software through Obfuscation: Can It Keep Pace with Progress in Code Analysis?, *ACM Comput. Surv.*, Vol.49, No.1, Article 4 (2016).

[2] Chandran, S., Syam, S.R., Sankaran, S., Pandey, T. and Achuthan, K.: From Static to AI-Driven Detection: A Comprehensive Review of Obfuscated Malware Techniques, *IEEE Access*, Vol.13, pp.74335–74358 (2025).

[3] Cohen, R., David, R., Yger, F. and Rossi, F.: Identifying obfuscated code through graph-based semantic analysis of binary code, *Applied Network Science*, Vol.10, No.1, Article 49 (2025).

[4] Greco, C., Ianni, M., Guzzo, A. and Fortino, G.: Enabling Obfuscation Detection in Binary Software through eXplainable AI, *IEEE Trans. Emerg. Top. Comput.* (online), DOI: 10.1109/TETC.2024.3439884 (2024).

[5] Chen, T. and Guestrin, C.: XGBoost: A Scalable Tree Boosting System, *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, pp.785–794 (2016).

[6] Tigress: The Tigress C Obfuscator (online), available from `https://tigress.wtf/` (accessed 2026-08-19).

[7] Zhao, Y., Tang, Z., Ye, G., Peng, D., Fang, D., Chen, X. and Wang, Z.: Semantics-aware Obfuscation Scheme Prediction for Binary, *Computers & Security*, Vol.99, Article 102072 (2020).

[8] National Security Agency: Ghidra - Software Reverse Engineering Framework, GitHub (online), available from `https://github.com/NationalSecurityAgency/ghidra` (accessed 2026-08-19).

[9] Salton, G. and Buckley, C.: Term-weighting approaches in automatic text retrieval, *Information Processing & Management*, Vol.24, No.5, pp.513–523 (1988).

[10] Quarkslab: Diffing Obfuscation Dataset, GitHub (online), available from `https://github.com/quarkslab/diffing_obfuscation_dataset` (accessed 2026-08-19).

[11] Jiang, S., Hong, Y., Fu, C., Qian, Y. and Han, L.: Function-level obfuscation detection method based on graph convolutional networks, *Journal of Information Security and Applications*, Vol.61, Article 102953 (2021).

[12] Lundberg, S.M., Erion, G., Chen, H., DeGrave, A., Prutkin, J.M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N. and Lee, S.I.: From Local Explanations to Global Understanding with Explainable AI for Trees, *Nat. Mach. Intell.*, Vol.2, No.1, pp.56–67 (2020).

[13] Lundberg, S.M. and Lee, S.-I.: A unified approach to interpreting model predictions, *Proc. 31st International Conference on Neural Information Processing Systems (NIPS '17)*, pp.4768–4777, Curran Associates Inc. (2017).

[14] MITRE ATT&CK: Obfuscated Files or Information, MITRE ATT&CK (online), available from `https://attack.mitre.org/techniques/T1027/` (accessed 2026-08-19).

© 2026 Information Processing Society of Japan